PROJECT DOCUMENTATION

Codex sandbox

Rootless Podman / Codex CLI

Contents

AI-assisted development . . . 2
Requirements . . . 2
Usage . . . 2
  Launch Codex . . . 2
  Just recipes . . . 3
Image construction . . . 4
Custom home . . . 4
  Global instructions and skills . . . 4
Tools . . . 5
  Codex CLI . . . 6
  Node.js and npm . . . 6
  Python with uv . . . 6
  Repeater . . . 7
  Go . . . 7
  Java with SDKMAN . . . 8
  Ansible authoring and validation . . . 8
  Lean 4 with Elan . . . 8
  Publishing . . . 9
Filesystem and persistence . . . 9
Isolation boundary . . . 10
  Network boundary . . . 10
Remove persistent state . . . 10
Releases . . . 10

A personal development environment for Codex CLI, powered by rootless Podman.

It keeps the container boundary and Codex's own Linux sandbox enabled: Podman exposes only explicitly selected working directories, and Codex runs generated commands through `bubblewrap` and seccomp in `workspace-write` mode.

This README is also available as a PDF.

AI-assisted development

Development is assisted by Codex CLI under close human direction. Each new version of `codex-sandbox` is developed from within the current version, so the project also exercises the environment it builds.

Requirements

- Rootless Podman
- Bash and standard GNU command-line tools
- Unprivileged user namespaces on the host
- `just` on the host if you want to use the repository recipes
- ShellCheck on the host if you want to run `just check`

Tools used inside the sandbox are installed in the image; host installations are not exposed to the container.

Usage

Launch Codex

Pass one or more host working directories to the launcher:

```
~/src/codex-sandbox/run.bash ~/src/my-project
```

For a shorter command, create a symlink from a directory on the host's `PATH`:

```
ln --symbolic ~/src/codex-sandbox/run.bash ~/bin/sbx
sbx ~/src/my-project
```

The image is built automatically the first time. Codex starts in `/workspace`, with each selected directory mounted below it using its basename. For example:

```
~/src/codex-sandbox/run.bash ~/src/frontend ~/src/backend
```

This creates `/workspace/frontend` and `/workspace/backend`; neither directory is treated as primary. Selected directory basenames must be unique within a launch.

Paths containing spaces, commas, or double quotes are supported; the launcher quotes the mount fields for Podman's CSV parser.

The launcher always passes `--no-daemon` to Codex. Each container therefore runs its CLI session directly instead of starting or using the shared app-server daemon. Normal interactive work and saved-session commands such as `resume` and `fork` remain available.

To launch with an empty, disposable workspace and no host working directory mounted, use the explicit `--empty-workspace` option:

```
~/src/codex-sandbox/run.bash --empty-workspace
```

The launcher never infers a working directory from the current directory. Calling it without either `--empty-workspace` or at least one directory is an error. Arguments after `--` can also be used in empty-workspace mode.

Arguments after `--` are passed to Codex:

```
~/src/codex-sandbox/run.bash ~/src/my-project -- --model gpt-6.1-sol
```

Building and launching are separate operations. To build or refresh the image without launching Codex, run `--build` by itself:

```
~/src/codex-sandbox/run.bash --build
```

Just recipes

The `Justfile` wraps the common repository workflows. Run `just` at any time to list the available recipes.

Recipe	Action
`check`	Run Bash syntax checks, ShellCheck, launcher help, and `git diff --check`
`configure-signing`	Configure this clone to sign commits with the owner's GPG key
`pdf`	Render this README as a styled PDF through Pandoc and Typst
`build`	Build or refresh the image without launching Codex
`run ARGS...`	Forward arguments to `run.bash` and launch Codex
`containers`	List running containers created from the sandbox image
`shell [CONTAINER]`	Open a login Bash shell in a running sandbox container

For example:

```
just check                      # Run static validation
just configure-signing          # Configure signed commits in this clone
just pdf                        # Render docs/codex-sandbox.pdf
just build                      # Build or refresh without launching
just run --empty-workspace      # Launch without host workspace mounts
just run ~/src/my-project       # Launch Codex
just containers                 # List running sandbox containers
just shell                      # Open Bash in the running container
```

Run `just shell` from another terminal while Codex is running. If several sandbox containers are active, select one explicitly:

```
just shell CONTAINER_ID_OR_NAME
```

Arguments to `just run` are forwarded to `run.bash`, including the separator for Codex arguments:

```
just run ~/src/my-project -- --model gpt-6.1-sol
```

Image construction

The image is built from `debian:trixie-slim` and tagged `codex-sandbox:local`. The `Containerfile`:

1. Creates the unprivileged `codex` account with UID/GID 1000, preserving compatibility with existing persistent home volumes.
2. Installs the Debian-packaged system and development tools.
3. Installs the pinned, checksum-verified official Node.js LTS archive with its bundled npm, then the pinned ast-grep release.
4. Installs the pinned, checksum-verified standalone Codex CLI package.
5. Installs uv from a versioned, checksum-verified installer.
6. Installs the checksum-verified official Go toolchain for the image architecture.
7. Installs the checksum-verified official Repeater CLI for the image architecture.
8. Installs the checksum-verified official Typst CLI for the image architecture.
9. Creates a checksum-verified SDKMAN seed for new home volumes.
10. Copies the repository's custom home and entrypoint into the image.
11. Switches permanently to the unprivileged `codex` user.

The resulting container root filesystem is read-only at runtime. Writable locations are mounted explicitly by `run.bash`.

To build or refresh the image without launching Codex:

```
just build
# or
./run.bash --build
```

Every build uses Podman's `--pull=newer` policy. This checks for a newer `debian:trixie-slim` base so current Debian package updates are incorporated when available. Node.js and the other explicitly pinned tools are updated by changing their build arguments and checksums.

Custom home

The repository's sparse `home/` directory is the image-managed portion of `/home/codex`.

During the image build, this tree is copied to `/usr/local/share/codex-sandbox/home`. At every container start, the entrypoint overlays it onto `/home/codex`. Matching paths are refreshed from the image; files absent from the repository template are left untouched.

This separates two kinds of home data:

- Repository-managed configuration comes from `home/` and changes through an image rebuild.
- Runtime state such as Codex credentials, sessions, user-installed tools, and downloaded toolchains remains in the persistent `codex-sandbox-home` volume.

Removing a path from `home/` does not remove a copy already present in the persistent volume. Deletion requires an explicit migration or recreation of the volume.

Global instructions and skills

The custom home provides a natural place for global Codex instructions and deliberate personal skills:

```
home/
├── .agents/
│   └── skills/
│       ├── build-silverbullet-wiki/
```

```
│       │   ├── SKILL.md
│       │   ├── agents/openai.yaml
│       │   └── assets/repeater-style.md
│       └── publish-project-pdf/
│           ├── SKILL.md
│           ├── agents/openai.yaml
│           ├── assets/theme.typ
│           └── references/visual-review.md
└── .codex/
    └── AGENTS.md
```

This makes `codex-sandbox` not only a development environment, but also a convenient Git-backed set of instructions for Codex.

The current `home/.codex/AGENTS.md` explains the workspace layout and installed tools to every Codex session and routes polished PDF requests to the personal skill. Project-level `AGENTS.md` files remain scoped to their own selected directories and take precedence where they apply.

The `publish-project-pdf` skill supplies the on-demand workflow for creating or refining polished PDFs from repository documentation. Its theme is a starter asset: Codex copies and adapts it only when a project does not already have a theme. The resulting repository owns its local copy and never imports the skill at build time, so its documentation remains reproducible outside this sandbox and does not change silently when the image is updated.

The PDF documentation for `codex-sandbox` was produced using this very skill.

The `build-silverbullet-wiki` skill supplies the conventions for creating and evolving focused SilverBullet knowledge wikis with Repeater cards integrated beside the knowledge they test. It covers wiki structure, Markdown conventions, card placement and display, parser-safe documentation, and validation. Guidance for writing pedagogically effective card content remains a separate concern.

Tools

Exact image-managed tool versions and checksums are recorded in `Containerfile`. In the update examples below, replace `<version>` with the release you intend to install.

The image includes the following user-facing tools:

Tool	Purpose	Installation
Codex CLI	Interactive coding agent	Verified official archive
Node.js	JavaScript runtime	Verified official archive
npm	JavaScript package manager	Bundled with Node.js
ast-grep	Syntax-aware search and rewriting	Pinned npm package
ripgrep (`rg`)	Fast textual search	Debian package
jq	JSON queries and transformations	Debian package
Ansible Core	Local playbook validation and reference	Debian package
Ansible Lint	Static checks for Ansible content	Debian package
yamllint	YAML syntax and style checks	Debian package
Git	Version control	Debian package
ShellCheck	Shell script analysis	Debian package
just	Repository command runner	Debian package
Pandoc	Markdown and document conversion	Debian package
Typst	Document typesetting and PDF generation	Verified official archive

uv	Python versions, environments, and tools	Verified installer
Go	Go compiler and development tools	Verified official archive
Repeater	Markdown spaced-repetition CLI	Verified official archive
SDKMAN	Java versions and SDKs	Verified seed installation
Elan	Lean toolchain manager	Debian package
curl, xz, zip	Downloads and archive handling	Debian packages

`bubblewrap` is also installed as a runtime dependency of Codex's Linux sandbox.

Codex CLI

The image installs the standalone Codex package for its build architecture. The version and both supported Linux archive checksums are explicit build arguments in `Containerfile`.

Before updating, open the GitHub release page for `rust-v$CODEX_VERSION` in the Codex releases and review its `codex-package_SHA256SUMS` asset. Then, update `CODEX_VERSION`, `CODEX_LINUX_AMD64_SHA256`, and `CODEX_LINUX_ARM64_SHA256` together.

A build fails before downloading Codex if the checksum selected for its architecture is still a placeholder or is not a lowercase 64-character SHA-256 value. After the download, the build also fails unless the archive's computed SHA-256 matches that recorded checksum exactly.

Node.js and npm

The build installs a pinned Node.js LTS release from the official Linux archive, with separate SHA-256 pins for AMD64 and ARM64. Node.js is an explicit development tool; the standalone Codex CLI does not require it.

The image uses the npm version bundled with that Node.js archive, keeping the runtime and package manager on the pairing distributed by the Node.js project.

Project dependencies remain in each selected project's `node_modules` directory. At runtime, global npm packages install below `/home/codex/.local`, whose `bin` directory is on `PATH`:

```
npm install
npm install --global npm-check-updates
```

The npm download cache lives below `/tmp/codex-npm-cache` and is removed with the container. Package downloads may require network approval, and persistent global installation may require filesystem approval when Codex runs these commands. Image-managed global packages remain installed system-wide.

Updating Node.js and npm

Review the intended Node.js LTS release and its official checksum file:

```
NODE_VERSION='<version>'
curl --fail --show-error --silent --location \
  "https://nodejs.org/dist/v${NODE_VERSION}/SHASUMS256.txt" \
  | rg "node-v${NODE_VERSION}-linux-(x64|arm64)\\.tar\\.xz$"
```

Update `NODE_VERSION`, `NODE_LINUX_AMD64_SHA256`, and `NODE_LINUX_ARM64_SHA256` together in `Containerfile`, then rebuild. npm updates alongside Node.js through the release archive. Node.js security updates require an explicit pin update; pulling a newer Debian base does not update it.

Python with uv

`uv` reads a project's `pyproject.toml`, `uv.lock`, and `.python-version` files and installs a matching Python version when needed:

```
uv sync
uv run python --version
```

Python versions and installed tools live below `/home/codex`, so they survive container and image replacement. The download cache lives below `/tmp/codex-uv-cache` and is removed with the container. Project virtual environments

remain in their selected working directories. `UV_LINK_MODE` is set to `copy` because the cache and project environments are on different filesystems.

Python and package downloads may require network approval, and persistent Python or tool installation may require filesystem approval when Codex runs these commands.

Updating uv

The image pins both the uv release and the exact SHA-256 of its architecture-aware installer. To prepare an update:

```
UV_VERSION='<version>'
curl --fail --show-error --silent --location \
  "https://astral.sh/uv/${UV_VERSION}/install.sh" \
  --output /tmp/install-uv.sh
grep '^APP_VERSION=' /tmp/install-uv.sh
sha256sum /tmp/install-uv.sh
```

Update `UV_VERSION` and `UV_INSTALLER_SHA256` together in `Containerfile`, review the installer changes, and rebuild the image.

Repeater

Repeater is installed system-wide from the official prebuilt archive for the image architecture. It can validate Markdown cards without a temporary download or a user-global installation:

```
repeater --version
repeater check . --plain
```

Updating Repeater

The image pins the Repeater version and the official SHA-256 digests for both supported Linux architectures. Obtain the release asset metadata from GitHub before an update:

```
REPEATER_VERSION='<version>'
curl --fail --show-error --silent --location \
  "https://api.github.com/repos/shaankhosla/repeater/releases/tags/v${REPEATER_VERSION}" \
  | jq -r '.assets[] | select(.name | test("^repeater-(aarch64|x86_64)-unknown-linux-gnu\
\.tar\\.xz$")) | [.name, .digest] | @tsv'
```

Update `REPEATER_VERSION`, `REPEATER_LINUX_AMD64_SHA256`, and `REPEATER_LINUX_ARM64_SHA256` together in `Containerfile`, then rebuild the image.

Go

The official Go toolchain is installed system-wide at `/usr/local/go`. `GOPATH` is `/home/codex/go`, and its `bin` directory is on `PATH`, so tools installed with `go install` are immediately available and survive container replacement:

```
go version
go test ./...
go install golang.org/x/tools/gopls@latest
```

Module and build caches live below `/tmp/codex-go-cache` and are removed with the container. Installed tools live below `/home/codex/go` in the persistent volume. Project source and vendor directories remain in the selected working directory. Downloads may require network approval, and persistent tool installation may require filesystem approval when Codex runs these commands.

Updating Go

The image pins the Go version and the official SHA-256 checksums for both supported Linux architectures. Obtain the current release metadata from Go's official download endpoint:

```
curl --fail --show-error --silent --location \
  "https://go.dev/dl/?mode=json" \
  | jq '.[0] | {version, files: [.files[] | select(.os == "linux" and (.arch == "amd64"
or .arch == "arm64")) | {arch, sha256}]}'
```

Update `GO_VERSION`, `GO_LINUX_AMD64_SHA256`, and `GO_LINUX_ARM64_SHA256` together in `Containerfile`, then rebuild the image.

Java with SDKMAN

SDKMAN is seeded for new home volumes and loaded automatically for Bash commands. Its files and downloaded SDKs live in the persistent home volume, so installed JDKs survive container and image replacement:

```
sdk list java
sdk install java 21.0.12-tem
sdk install java 17.0.20-tem
sdk default java 21.0.12-tem
java -version
```

For a project-specific JDK, enter the project directory and create an SDKMAN environment file:

```
sdk env init
```

Edit `.sdkmanrc` to select the desired version, then activate it with `sdk env`. Listing or downloading SDKs may require network approval when Codex runs the command.

SDKMAN's automatic startup connectivity check is disabled because Codex commands do not have network access by default. The entrypoint enforces this setting for both new and existing home volumes.

Updating SDKMAN

The SDKMAN bootstrap URL is mutable. The image therefore pins the bootstrap script's exact SHA-256 along with the CLI and native component versions. A changed script fails the build instead of silently installing different code.

Confirm the intended release on the SDKMAN releases page, then inspect and hash the bootstrap script:

```
curl --fail --show-error --silent --location \
  "https://get.sdkman.io?rcupdate=false" \
  --output /tmp/install-sdkman.sh
grep '^export SDKMAN_\(VERSION\|NATIVE_VERSION\)=' /tmp/install-sdkman.sh
sha256sum /tmp/install-sdkman.sh
```

Update `SDKMAN_VERSION`, `SDKMAN_NATIVE_VERSION`, and `SDKMAN_INSTALLER_SHA256` together in `Containerfile`, review the script, and rebuild. SDKMAN's published GitHub checksum covers its release ZIP, not the bootstrap script, so it cannot be used as `SDKMAN_INSTALLER_SHA256`.

The image seed initializes SDKMAN only when its files are absent from the persistent home. Rebuilding does not replace an existing SDKMAN installation or its installed JDKs.

Ansible authoring and validation

The image includes Ansible Core, `ansible-lint`, and `yamllint` for authoring and locally validating playbooks, roles, and collections. Useful commands include:

```
ansible-playbook --syntax-check playbook.yml
ansible-lint
yamllint .
ansible-doc ansible.builtin.copy
ansible-config dump
```

Ansible's local temporary files are directed to `/tmp`, which remains writable inside Codex's sandbox and is removed with the container.

The image-managed Codex instructions prohibit Ansible commands that connect to or change remote hosts. This includes check mode, which still contacts managed hosts, and dynamic inventory queries that contact external services. Ansible is present as an authoring and validation tool, not as authorization to deploy.

Project-specific collections remain declared in each project's `requirements.yml` and can be installed with `ansible-galaxy` when requested. Collection downloads may require network approval.

Lean 4 with Elan

Elan and its `lean` and `lake` proxies are installed system-wide. Downloaded Lean toolchains live in the persistent home volume. Inside a Lean project, `lean` and `lake` select the version from `lean-toolchain` and download it when necessary:

```
lake build
lean Main.lean
```

Outside a project with a `lean-toolchain` file, select a default first:

```
elan default stable
lean --version
```

Toolchain and Lake dependency downloads may require network approval when Codex runs them.

Publishing

Pandoc can produce standalone HTML, DOCX, EPUB, and other document formats directly from Markdown. Typst provides PDF typesetting without a LaTeX installation. For example:

```
pandoc report.md --pdf-engine=typst --output=report.pdf
```

Typst can also compile `.typ` sources directly:

```
typst compile report.typ report.pdf
```

Typst Universe packages are downloaded on demand and cached in the persistent home volume. Their first use may require network approval. Prefer project-local templates when a build must work offline or use only repository-reviewed inputs.

Updating Typst

The image pins the Typst version and official GitHub release SHA-256 digests for both supported Linux architectures. Query the intended release metadata before an update:

```
TYPST_VERSION='<version>'
curl --fail --show-error --silent --location \
  "https://api.github.com/repos/typst/typst/releases/tags/v${TYPST_VERSION}" \
  | jq -r '.assets[] | select(.name | test("(aarch64|x86_64)-unknown-linux-musl\\.tar\
\.xz$")) | [.name, .digest] | @tsv'
```

Update `TYPST_VERSION`, `TYPST_LINUX_AMD64_SHA256`, and `TYPST_LINUX_ARM64_SHA256` together in `Containerfile`, then rebuild the image.

Filesystem and persistence

Location	Backing	Lifetime
Selected `/workspace/NAME` children	Optional host bind mounts	Persists on the host
`/workspace` root	1 GiB private tmpfs	Removed with the container
`/home/codex`	`codex-sandbox-home` volume	Persists across containers
`/tmp`	1 GiB private tmpfs	Removed with the container
`/run`	64 MiB private tmpfs	Removed with the container
Container root	Read-only image	Replaced with the container

Files written directly to `/workspace` disappear when the container exits. Only changes inside selected child directories persist on the host. In `--empty-workspace` mode, no host working directory is mounted and every file created below `/workspace` is disposable.

The home volume contains Codex login and session state, user-global npm packages, uv-managed Python versions and tools, Go-installed tools, cached Typst Universe packages, SDKMAN and installed JDKs, and Elan-managed Lean toolchains. The container itself is launched with `--rm`; only this volume and selected host directories outlive it.

Isolation boundary

The outer Podman container:

- Runs rootless with `--userns=keep-id`.
- Drops all Linux capabilities and enables `no-new-privileges`.
- Uses a read-only root filesystem and a 512-process limit.
- Mounts only explicitly selected host directories, if any, and its private home volume.
- Does not mount the host home, SSH configuration, environment, or Podman API socket.
- Uses private SELinux relabeling for selected working directories.

Podman's minimal init process runs as PID 1 to forward signals and reap orphaned children. Within this boundary, Codex retains its normal `bubblewrap~/seccomp sandbox with ~workspace-write` access and approval on demand.

If commands fail with a `bwrap` or namespace error, do not disable the Codex sandbox. The host's rootless Podman configuration must support unprivileged user namespaces for both isolation layers to work.

Network boundary

Codex needs network access to communicate with OpenAI, so the outer container has Podman's normal outbound network access. Commands spawned by Codex remain subject to its `workspace-write` network restrictions and approval policy. No outer egress allowlist is currently configured.

Remove persistent state

After every sandbox container has stopped, the persistent home can be removed:

```
podman volume rm codex-sandbox-home
```

This permanently deletes Codex credentials and sessions, user-global npm packages, uv-managed Python versions and tools, Go-installed tools, cached Typst Universe packages, SDKMAN and its installed JDKs, and Elan's downloaded Lean toolchains and configuration. It does not affect selected host working directories.

Releases

Releases are created manually through the `Release` workflow in GitHub Actions. Run the workflow from `master` and choose a `patch`, `minor`, or `major` version increment.

The workflow finds the latest stable `vMAJOR.MINOR.PATCH` tag, applies the selected increment, creates the resulting tag at the selected `master` commit, and creates a GitHub release with generated notes and the tracked PDF from that commit. The release asset is named `codex-sandbox-vMAJOR.MINOR.PATCH.pdf`. If the repository has no version tags yet, the calculation starts at `v0.0.0`.

Pushes and pull requests do not trigger releases. The workflow does not publish packages or container images.